



The chatbot invented a policy — the company pays.

Moffatt v. Air Canada · BC tribunal · 02/14/2024

1

A passenger asked the chatbot about a bereavement fare

2

Bot: “buy at full price, get the difference back within 90 days”

3

The real policy did not allow this — and it was on the very page the bot linked to

4

Tribunal: “the bot is not a separate legal entity” → the company refunds the difference^[1,2]

This is not a model glitch — it is the wrong architecture choice for the task. This whole lecture is about that class of error.

^[1] McCarthy Tétrault — *Moffatt v. Air Canada* (BC CRT, 14.02.2024) · ^[2] ABA Business Law Today — companies are liable for their AI chatbot

LECTURE 3

AI-system architectures: agents, RAG, API

03

Which architecture to choose for the task —
and when the right answer is “no AI”

Lecture route — six sections.

One keystone line: choosing the architecture for the task — and when the right answer is “no AI”.

0



Opening

Air Canada: the wrong architecture costs money

1



The prompt and its limits

what a single call can do and where its ceiling is

2



RAG

external knowledge into context — and where it quietly breaks

3



Fine-tune

changing weights for behavior, not for knowledge

4



Agents

loop, harness, memory, security — and failures

5

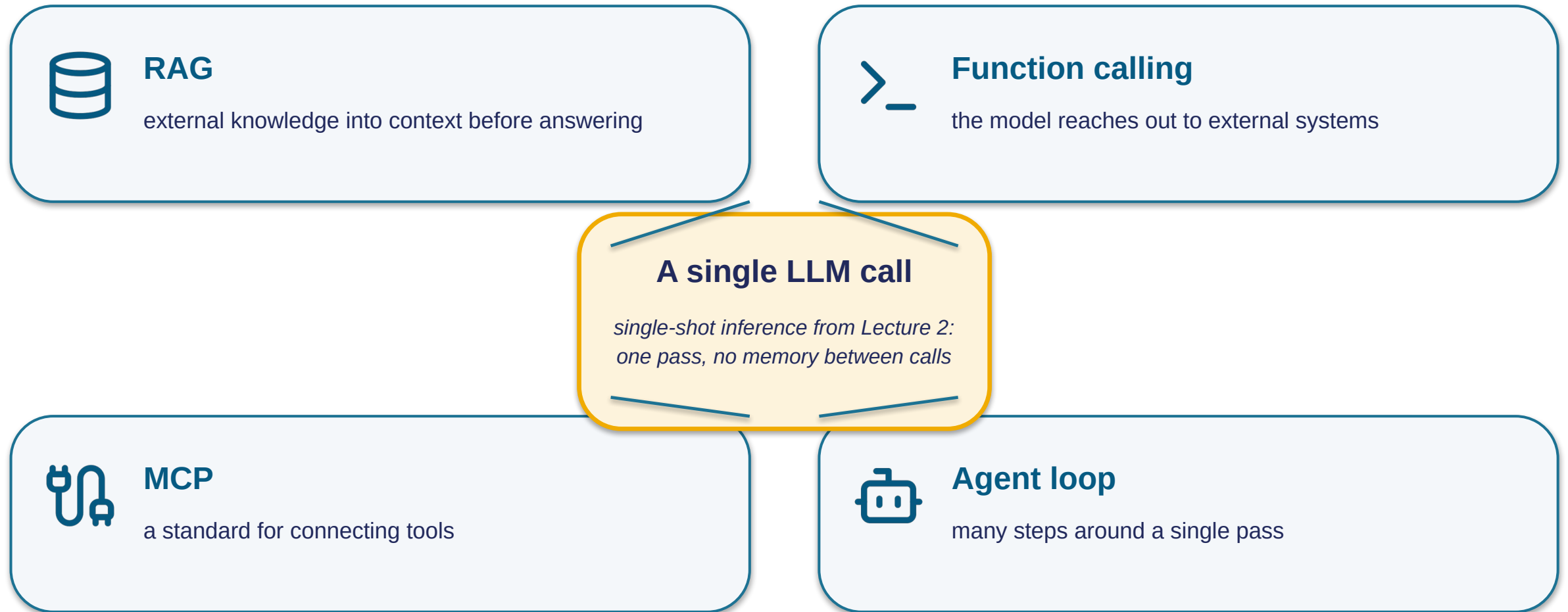


Framework

ladder + checklist: how to choose fast and with justification

What we carry over from Lecture 2 — and what we build on top.

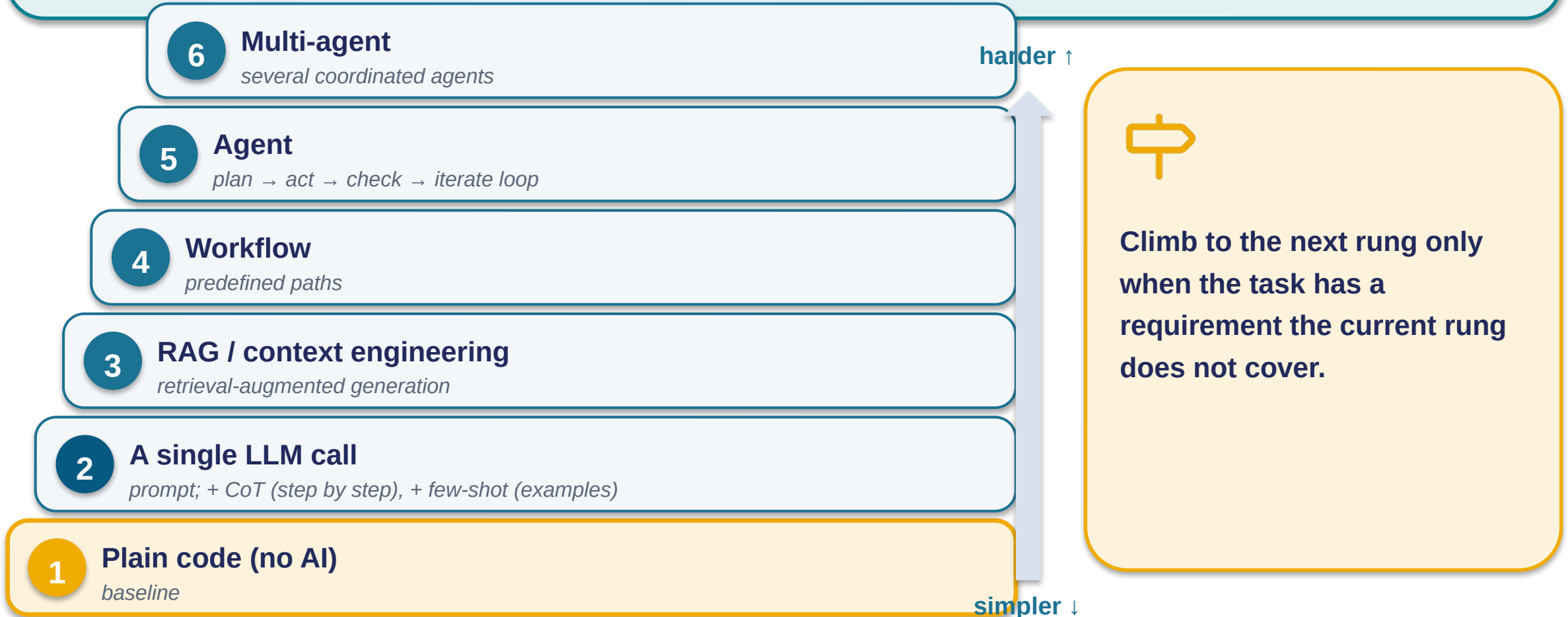
Four wrappers are built around a single model call. We do not re-explain two ready blocks from Lecture 2.



Carried over ready from Lecture 2: single-shot inference and semantic search on embeddings (the basis of RAG). The details of each wrapper come later, section by section.

THE CENTRAL QUESTION OF THE LECTURE

I have a task and access to an LLM. Which architecture do I choose — and when is the right answer “no AI”?



The ladder is a map of the lecture, not a demand to understand everything now. We cover each rung separately.

SECTION 1

The prompt and its limits

We've seen the whole ladder — now from the bottom: what a single call can do and where its ceiling is, before complicating anything.



Section 0
Opening

Section 1
Prompt

Section 2
RAG

Section 3
Fine-tune

Section 4
Agents

Section 5
Framework

The default — a single call with a good prompt.

The model knows only what is in the prompt (plus what was in the weights at training time). It has nowhere else to draw from.



A single LLM call with a good prompt

- minimal cost (one pass)
- minimal latency (no extra round-trips)
- maximum predictability (no loops, no retrieval that quietly degrades)

Add RAG →

an indexing pipeline + a vector store + a retrieval component (can silently degrade) + metrics for its quality

Add tools / a loop →

external calls (they fail, they stall) + loops (they diverge) + trajectory nondeterminism + a new attack surface

Do not complicate the architecture without a reason expressed in the task's requirements. This is placing the burden of proof, not primitivism.

A role in the prompt tunes the tone — not the accuracy.

“You are an experienced lawyer” shifts the model's attention (the mechanism — Lecture 2) toward text that resembles such a role's answer — but that is about style, not about facts.

The common myth

“I'll write an expert role — and the model will answer facts more accurately”

What the experiment showed

162 personas · 6 relationship types · 8 domains · 2410 factual QA questions · 4 LLM families. Personas did NOT improve answer accuracy compared with answering with no persona at all^[1].



What a role actually affects

the tone and depth of the exposition — how formally and thoroughly, not whether a fact is correct (the effect is model-specific).

Need factual accuracy? The tool is not the wording of a role, but sound context and, where needed, RAG (grounding in a verifiable source). This is one more “magic pill” refuted by measurement.

[1] [Zheng et al. 2024, Findings of EMNLP — personas don't improve factual accuracy](#) · Zheng, Pei, Logeswaran, Lee, Jurgens · Findings of EMNLP 2024 (arXiv:2311.10054) + arXiv:2605.29420 (2026).

Prompt structure: separate instruction, context, data.

A flat prompt forces the model to guess where the instruction ends and the data begins. An explicit boundary removes that ambiguity.



Instruction

what to do



Context

on what basis



Data

what exactly to work with

Delimiters

- XML tags: `<instruction>...</instruction>`
- Markdown headings: `## Task · ## Data`
- Triple quotes / backticks around the data

The same principle as structured output (Section 4): there the schema sets the structure of the OUTPUT, here delimiters set the structure of the INPUT. Clear structure instead of “infer the format from meaning.”

A model that has been shown where the instruction ends and the data begins less often mistakes a fragment of data for a new command — the same confusion underlies prompt injection (Section 4).

Chain-of-thought helps — but it cannot be audited.

Without CoT

“There were 23 apples, 7 went bad, we bought 2 more crates of 6.
How many good ones?”

→ a plausible but wrong number

With CoT (“solve step by step”)

$23 - 7 = 16^{[1]}$ · $2 \times 6 = 12$

$16 + 12 = 28$

→ correct

Technically this is still a single call — CoT is a tool for a class of tasks (a chain of steps), not a global switch.

But the spoken-out reasoning is not obliged to reflect the real cause of the answer (low faithfulness):

~25%

Claude 3.7 Sonnet

~39%

DeepSeek R1

— the share of cases where the model mentioned the hint it actually used^[2].

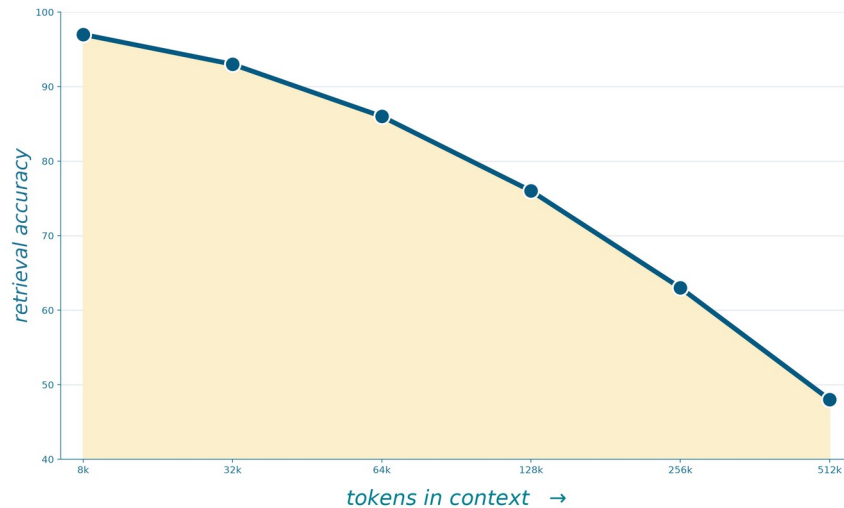
Control based on the model's self-explanation is not control. A human validator checks the **RESULT** and the facts against an external source, not the plausibility of the text. And this is also why the check step in the agent loop (Section 4) must not be the model's self-assessment.

Context engineering: the minimum of high-signal.

Prompt engineering is a single instruction. Context engineering is curating the whole set of tokens the model sees at inference.

Retrieval accuracy ↓ as the context grows

SCHEMATIC — illustration of the effect, not measured data · effect: Chroma Research, 2025



When NOT RAG (point 1)

a small, stable corpus that fits in the window →
full-context + prefix caching, not RAG
infrastructure



RAG here would add fragility with no gain

context rot = the same “lost in the middle” from L2 — a new term, not a new phenomenon ^[1,2]

“Find the smallest set of high-signal tokens that maximizes the probability of the desired outcome” — that is an engineering requirement, not aesthetics ^[3].

Cheat sheet: how to build a prompt.

The minimum below which a prompt systematically underperforms. Apply it to any prompt before the first run.

1 Role

if you need tone/register — and NOT as a promise of accuracy

2 Task

a concrete verifiable action, not a vague wish

3 Context

the minimum necessary, not “everything that might come in handy”

4 Output format

stated explicitly if the answer is machine-processed

5 Delimiters

if there is more than one kind of content (instruction/data)

6 Examples (few-shot)

only if the format is not obvious from a single instruction

7 CoT

only if the task requires multi-step reasoning

8 Length

no longer than necessary — extra tokens “drown” in the context

A compact form of the whole section: role $\neq$ accuracy, structure helps separate inputs, CoT is a targeted tool, context is minimal. For big architectural decisions (RAG / FT / agent) — the eight-step checklist of Section 5.

SECTION 2

RAG: retrieval-augmented generation

Retrieve what's relevant → put it in context → answer grounded in the source



Section 0
Opening

Section 1
Prompt

Section 2
RAG

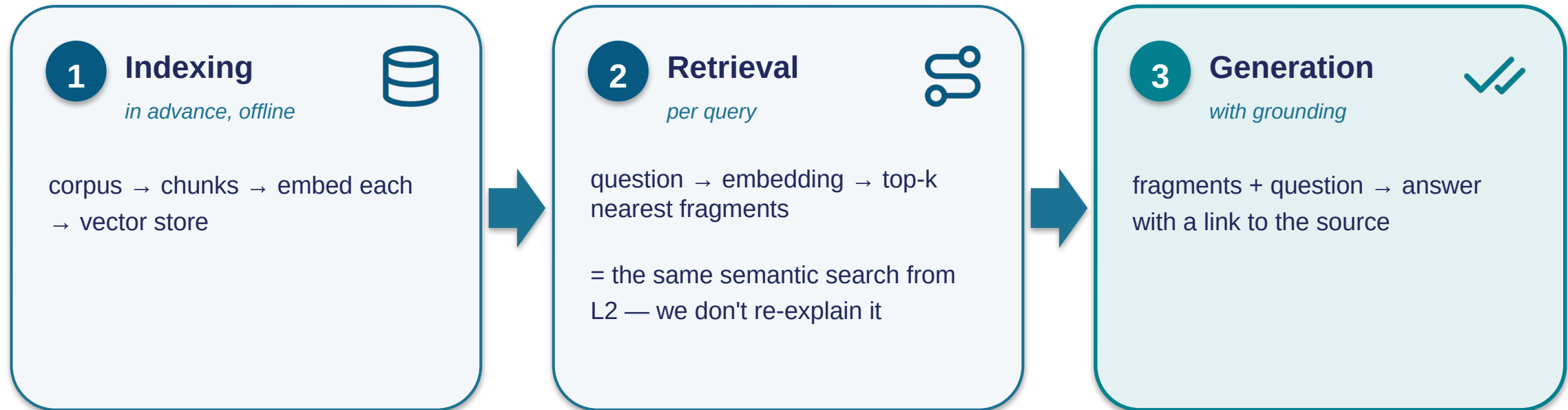
Section 3
Fine-tune

Section 4
Agents

Section 5
Framework

The RAG principle — three steps.

RAG = indexing → retrieval (semantic search from L2) → grounded generation; “I don't know” is a correct answer.



“I don't know” / “see source X” is a correct answer from a RAG system^[1]. A plausible answer on irrelevant retrieval is a defect, not “better than nothing.”

[1] [Lewis et al. 2020 — Retrieval-Augmented Generation \(NeurIPS\)](#)

When RAG is the right choice.

RAG is justified when there is a strong signal on the features below^[1] — and no blockers from the next slide, “when NOT RAG.”

Large / growing

does not fit in the window as a whole, or it is expensive to put the entire corpus into every request

Changing

documents, prices, regulations update more often than model versions ship

Freshness + provenance

the answer relies on a verifiable source; you can show where a fact came from

Private base

the company's knowledge is not in the weights of a public model

The features reinforce each other → RAG

A corporate base of thousands of regulations, updated weekly, an answer with a mandatory link to the clause: all features converge, no simpler mechanism covers them jointly → a textbook RAG profile.



A single feature is a reason to look closer^[2], not to build automatically: check against the blockers on the next slide.

When RAG is NOT the right choice.

Knowing when RAG is not needed is more valuable: it is a fashionable architecture, and people put it where it does harm.

1 The corpus fits in the window

a rough guide — under ~200k tokens, rarely changing

→ **full-context + prefix caching, not RAG infrastructure**

2 A fixed policy / value^[2]

a fare, a price, a regulation clause, a rule

→ **a deterministic lookup / a static page**

3 Data is available live via API / MCP

in an internal service, database, or another system's search

→ **call the tool directly; a RAG index on top is an extra, more fragile, and more stale layer**

RAG is redundant if ANY of the three holds^[1]: (a) the corpus fits in the window and is stable, (b) the task reduces to a fixed value, (c) the knowledge is already available directly and live through a tool. “A direct call returns data as of the request; a RAG index — as of the last indexing.”

RAG failure at scale.

“Returned something” ≠ “returned the right thing.” RAG has no ^[1,2] “not found” signal — it always returns the k nearest, even irrelevant ones.



Legal-AI

the “nearest” cases are from another jurisdiction / an overturned precedent — the model relies on them as fact



Medical-RAG

mixed fragments from different patients — close in symptoms, clinically impossible to combine



Support bot

ran on hundreds of articles; after growing to thousands, quality quietly dropped — no one noticed

Air Canada — architecture breakdown^[3]

Diagnosis

generated plausible text was put in a role that required a retrieved, verified fact — a grounding failure

The right alternative

a fixed policy → lookup / page; if a dialogue is needed → RAG with strict grounding, a mandatory citation, an explicit “I don't know,” and human review

SECTION 3

Fine-tune vs prompt vs RAG

We solved the knowledge problem through RAG. But what if the problem is not knowledge, but the model's behavior — its tone, format, policy?



Section 0
Opening

Section 1
Prompt

Section 2
RAG

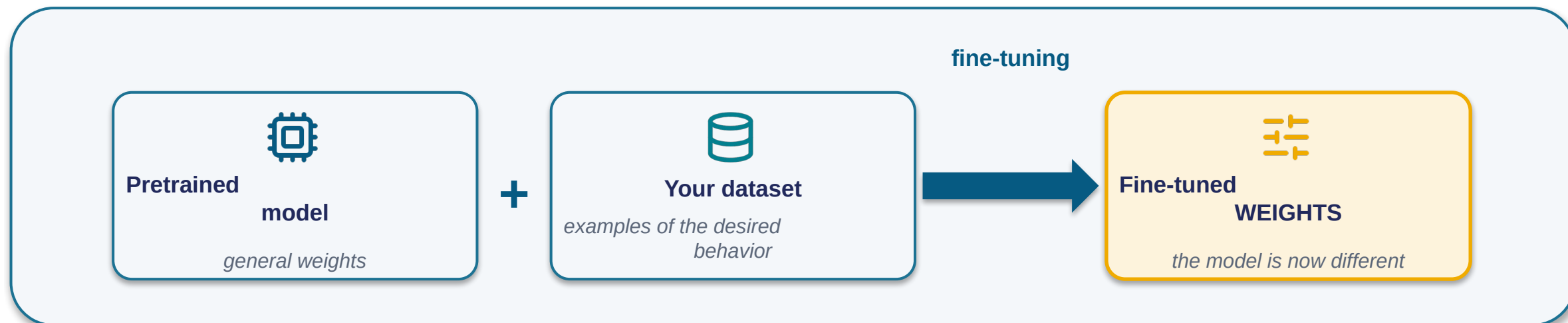
Section 3
Fine-tune

Section 4
Agents

Section 5
Framework

What fine-tuning is.

Fine-tuning is continuing the training of an already-built model on your data. In L1 — a type of use; here — an architectural choice, one of the rungs of the ladder.



Prompt / RAG → CONTEXT

They change only the input — the weights aren't touched; the effect lives only within the request.

Fine-tuning → THE WEIGHTS THEMSELVES

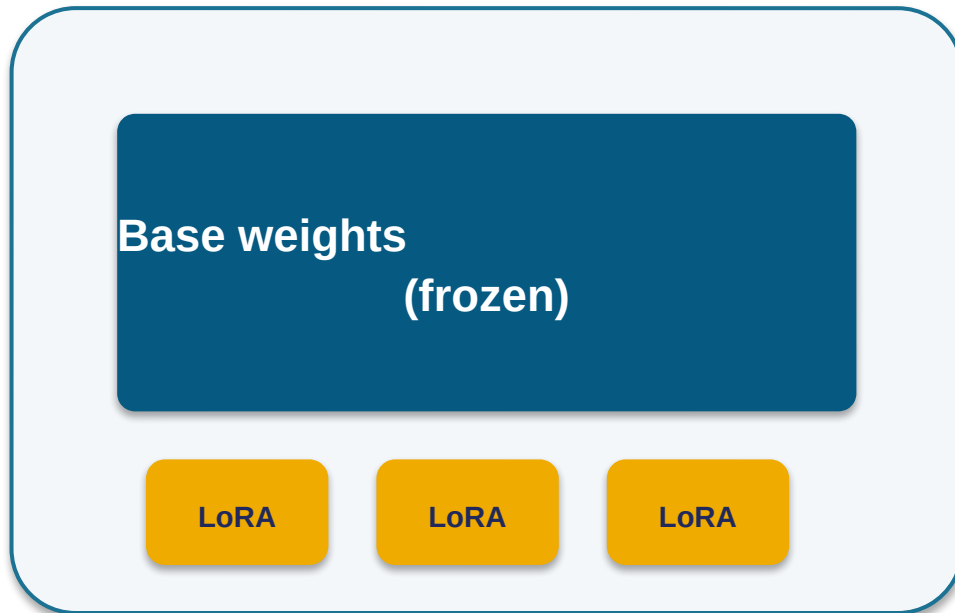
The change is built into the model — it always applies and costs more than what changes the context.

Prompt/RAG = “what to show the model.” Fine-tuning = “change the model itself^[1].” In practice “fine-tune” almost always means LoRA/PEFT, not retraining all the weights (next slide).

[1] IBM — RAG vs Fine-Tuning (fine-tuning changes the weights)

PEFT instead of full fine-tuning.

PEFT — the base weights are frozen, and only a small set of adapters is trained. LoRA — low-rank adapter matrices; QLoRA — the same on top of a quantized model^[1,2].



adapters — megabytes vs gigabytes

98.4% of models tagged PEFT are LoRA^[3]

out of 20,834 cards on the Hugging Face Hub · caveat: the share among models tagged PEFT, not among all fine-tuning

1. Cheaper and faster

millions of parameters trained instead of billions; QLoRA — on a single GPU

2. Modularity

adapters are megabytes vs gigabytes; one base — many specializations

3. ↓ Catastrophic-forgetting risk

the base is frozen, physically not rewritten under the new signal — an architectural argument

PEFT (LoRA/QLoRA) is almost always better than full fine-tuning: cheaper, more modular, ↓ forgetting risk. Full FT in 2026 — almost never.

Fine-tuning is not dead^[1] — it narrowed down to behavior.



Left fine-tuning (2026)

knowledge, facts, whatever changes → RAG / long context. Baking facts into weights: you can't cite them, update one point, or delete just one.



Stayed with fine-tuning

behavior · style · output format · policy adherence. Distillation is a separate technique, often applied in tandem with fine-tuning^[2,3].

If the task requires...	→ the right tool	...and NOT this one, because
knowledge changes / freshness, provenance needed	RAG (or long context for a small corpus)	not fine-tuning: it goes stale, retraining is expensive, forgetting risk
stable behavior / tone / format / policy	fine-tuning (PEFT)	not RAG: it supplies knowledge but doesn't change the model's manner
cut cost / latency on a narrow task	fine-tune teacher + distill student	two separate techniques in tandem, NOT “distillation = a kind of fine-tuning”
a deterministic, verifiable answer	plain code, no AI	neither RAG nor FT: AI would add nondeterminism with no gain

The real question in 2026 is not “RAG or fine-tuning,” but “what here is knowledge (→ RAG), what is behavior (→ PEFT), what is deterministic (→ code).” A hybrid is the norm where there is BOTH a knowledge problem AND a behavior problem.

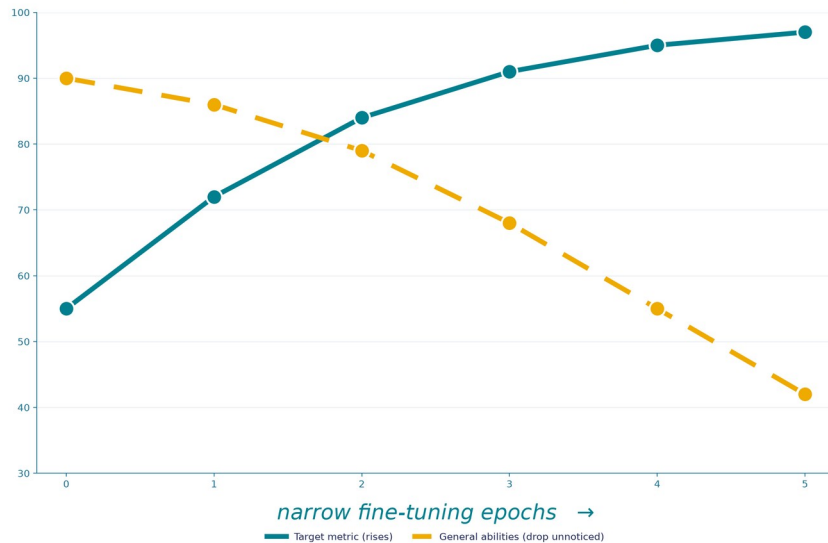
[1] [BigData Boutique — Fine-Tuning When RAG Isn't Enough](#) · [2] [Hinton, Vinyals, Dean 2015 — Distilling the Knowledge](#) · [3] [PEFT survey — taxonomy of tuning methods](#)

Failure: catastrophic forgetting.

Catastrophic forgetting — degradation of the model's general abilities as a result of narrow, aggressive fine-tuning.

Narrow aggressive fine-tuning: target ↑, general abilities ↓

SCHEMATIC — illustration of the effect, not measured data · effect: Luo et al., arXiv:2308.08747, 2023



No eval loop on general tasks + no dataset/weight versions → you won't see the breakage before prod + you won't be able to roll back → this is not a “risk,” it is a criterion for “DON'T fine-tune”

The right way: PEFT (frozen weights — lower risk); for changing knowledge — RAG, not FT at all.

worse as model scale grows — a large model has a higher starting level, so it has “more to fall”

SECTION 4

Agents

From an interlocutor in a chat window to a component of a production system: the loop, the harness, memory, tool access — and where all of it breaks.



Section 0
Opening

Section 1
Prompt

Section 2
RAG

Section 3
Fine-tune

Section 4
Agents

Section 5
Framework

The model becomes a system component: API + MCP.



Structured output

output strictly by schema (JSON), not text to parse

embeddable



Function calling

the model formulates “call X”; your code runs it, not the model

active



Prompt caching

don't recompute the unchanged prefix on every request

economical



MCP — “USB-C for LLM tools”

N×M incompatible integrations → **N+M**^[1]

a tool once = an MCP server; a model once = an MCP client

AI Anthropic
11/2024



 OpenAI
03/2025



 Google
04/2025

Standardizing the connection ≠ security of what's connected — and it aggravates the trust problem.

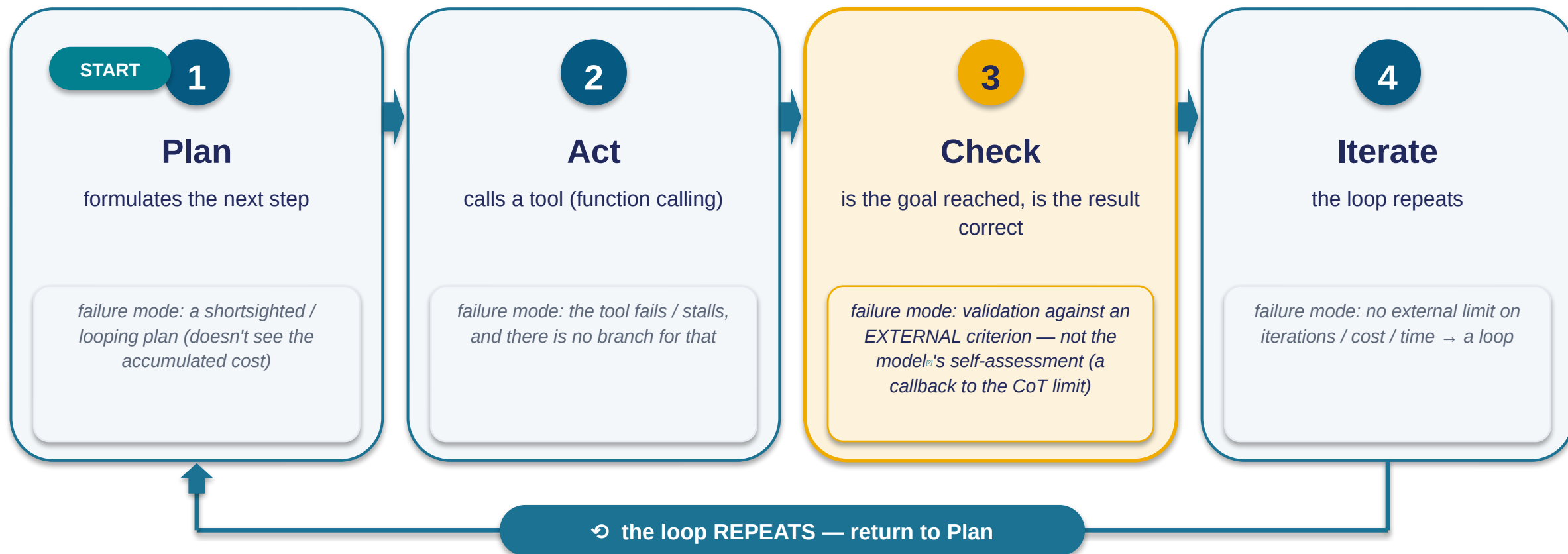
- code in your environment / access to data
- the description enters the context — a carrier for prompt injection

No mechanism makes the model more reliable — the ladder rule still holds. Ease of connection is not an argument for connecting.

[1] [Anthropic — MCP donation / Agentic AI Foundation \(N×M → N+M\)](#) · Up-to-date savings figures and the scale of the MCP ecosystem — in the chapter.

The agent loop: plan → act → check → iterate.

An agent is an architecture where the model doesn't make one pass but works in a loop, defining the sequence of steps itself^[1].



Workflow vs Agent.

A predictable task → workflow^[1]; unpredictable AND the value justifies a multiple increase → agent^[2].

Workflow

LLM and tools along paths predefined in code

- the sequence of steps is known in advance
- predictable, auditable
- most reliable production systems are workflows

Agent

the LLM dynamically defines its own process

- the sequence is not fixed in advance
- many times more tokens than a chat
- lower auditability, higher risk of loops

Can I write out the sequence of steps in advance, before running? yes (even with branches) → workflow · fundamentally no AND the value justifies multiple cost/risk → agent

Nesting is the norm: a code-review agent calls a “linter → tests → format” workflow as a single step; a ticket-processing workflow delegates parsing a free-form complaint to a mini-agent. Dynamics only where they're needed.

Find the simplest. “Too lazy to formalize” does not make a task unpredictable.

Every agent-harness slot is a trade-off, not an upgrade.

A real assistant agent (Claude Code, Cursor, Aider) is the plan → act → check → iterate loop PLUS a harness. Five typical slots^[1]:



Just as you don't climb the architecture ladder without a task requirement — don't add memory, subagents, or MCP to an agent “just in case.” Each slot carries its own cost: operational complexity, a new failure surface, a new trust boundary — and must answer a concrete trigger (Section 5).

^[1] agent-harness-registry — map of the agent's equipment slots

Agent memory — the same scale question as RAG.

Memory is what an agent remembers BETWEEN sessions (as opposed to the context of a single conversation). The spectrum runs from a flat file to graph databases^[1].

Flat file

the agent appends facts to a text log and reads it in full at startup. Works while the log is small and stable — a direct parallel to the RAG criterion “the corpus fits in the window.”



Vector / graph database

- mem0 — cross-session user memory
- Cognee — memory over a knowledge graph
- Graphiti / Zep — temporal graph: when a fact is true, when it went stale

The same knowledge-scale question that RAG answers for a document corpus arises here for the agent's own memory. Don't put a graph database on an agent with short, unconnected sessions — that is the same requirement-free technical debt as RAG for ten articles.

^[1] [agent-harness-registry \(live-eval\)](#) — the agent memory spectrum · Source: the public agent-harness-registry (workain lab, live-eval).

“An agent that remembers” — not always better.

Having memory intuitively seems like a pure improvement. An independent live-eval shows: sometimes it is dramatically not^[1].



Letta — Tier D

loses to BOTH the bare model AND the flat file on all tasks.
persistbench_v1:

 Bare model	1.000	94 s
 Flat file	0.833	159 s
 Letta	0.750	496 s

Mechanisms: capitulation under pressure · verbosity drowns the fact · the fact is noticed but not committed.

Caveat: the test is on Letta v0.6.7 — behind the current v0.16.8 (~18 mo).

Anthropic Memory Tool — Tier B

A strong result overall — but even it loses data on

17% of tasks

- an explicit refusal to record an “ephemeral” fact
- an unmotivated refusal as “out of scope”
- a silent lossy summarization — the detail cannot be recovered
- non-reproducibility: the same conversation twice → a different result

An instruction file for an agent is not a “magic inoculation.”

The operational layer — convention files (the CLAUDE.md / AGENTS.md class) + a task log. The intuition “I wrote an instruction → it's better” diverges from measurement.



presence paradox

RCT (Gloaguen et al. 2026): the mere presence of an instruction file gives NO significant gain^[1] in success — while cost and the number of steps grow.

It helps only where it actually fills a documentation gap.



Honest Lying

Dixit, Kamal, Oates 2026: self-authored memory can ENTRENCH a false belief — the log cements an early error instead of revising it.

The reflection is wrong → later attempts rely on it.



claude-code#51735

A real case: an error acknowledged in writing did NOT prevent it from recurring 25 days later^[2].

A written record of a failure ≠ a guarantee that behavior will change.

Useful when it fills a real gap; useless when it duplicates what's available to the model; harmful when self-authored memory entrenches an error. The same pattern as a role in the prompt: “add X → better” is not confirmed by measurement.

^[1] [Gloaguen et al. 2026 — Evaluating AGENTS.md \(presence paradox\)](#) · ^[2] [GitHub anthropics/claude-code#51735 \(the error repeated after 25 days\)](#)

Skills, subagents, access — and the trust boundary.



Skill

a reusable “how-to” procedure for a recurring task



Subagent

a separate context window: don't clutter the main one + isolate the untrusted



MCP access

each connection is a new trust boundary and retention policy



Security: as soon as an agent delegates and connects, a surface appears that a single call did not have ^[2,3]

GitHub MCP heist (May 2025)

an issue with an embedded instruction + an over-broad token (a PAT for all repos) → the assistant dumped private repositories into a public PR^[1].

Catastrophe = injection × broad rights. Remove either — the attack fails.

“We have ZDR” ≠ “stored nowhere”: a court order (NYT v. OpenAI) + third-party/MCP outside ZDR^[4]. Regulated data — don't send without ZDR/BAA.

4 design rules

1

Least-privilege

minimum tokens/rights

2

Isolate the untrusted

separate from privileges (subagent)

3

Human-in-the-loop on write

the irreversible — only through a human

4

Allowlist / pin

audited versions; deny-by-default

Real coding agents — through the harness lens.

The difference between the tools is not “model quality,” but which harness slots are filled and where the agent physically lives^[1].

<> Claude Code

CLI + IDE · proprietary

a broad harness: memory, instruction files, skills, full subagents, MCP — almost all 5 slots

the price is large operational complexity

<> Aider

CLI-first · open source

minimal simplicity: no developed memory, no subagents, no skills. ~47k★ on GitHub

the “thin” harness is a deliberate choice, not underdevelopment

<> Cursor

desktop IDE (a VS Code fork) · proprietary

an agent inside the editor — not a terminal. The form of integration is a separate axis from the harness

where the agent lives is also an architectural decision

<> OpenHands

self-hosted platform · MIT · ~80k★

broad autonomy, local/Docker/cloud deployment

a working hypothesis by profile match — a likely candidate for “OpenClaw,” not a confirmed fact

The choice of tool obeys the same rule: don't take the most-equipped one by default — look at which harness your specific task needs.

^[1] agent-harness-registry — a survey through the equipment frame

Agent failures.

Each failure is an agent-loop failure mode in a dated case: lesson + alternative.



1. A loop with no limits

An agent on “sync the orders” got an HTTP 429
→ plan → call → 429 → ...

\$4,200 over 63 hours^[1]

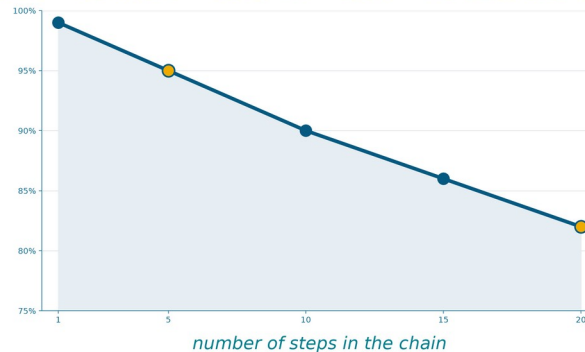
Baseline: a retry script with backoff would solve the same task in seconds-to-minutes, essentially for free

\$4,200 is the price not of automation, but of the wrong architecture choice for a predictable task

A more suitable architecture: a retry-with-backoff script, not an agent; budget and iteration limits **OUTSIDE** the agent

2. Compounding errors

5×99% ≈ 95%, 10 ≈ 90%, 20 ≈ 82% — reliabilities multiply



Takeaway: “improve the step” is a weak lever; “fewer hops + validation between steps^[2]” is a strong one



3. Multi-agent fragility

dependent subtasks → parallel subagents make conflicting^[3] implicit decisions

A more suitable one: a single-threaded linear agent; multi-agent only for broadly-parallel, independent work

SECTION 5

How to choose: the decision framework

We've covered all the architectures separately — and where each one fails. Now let's assemble it into a single decision tool.



Section 0
Opening

Section 1
Prompt

Section 2
RAG

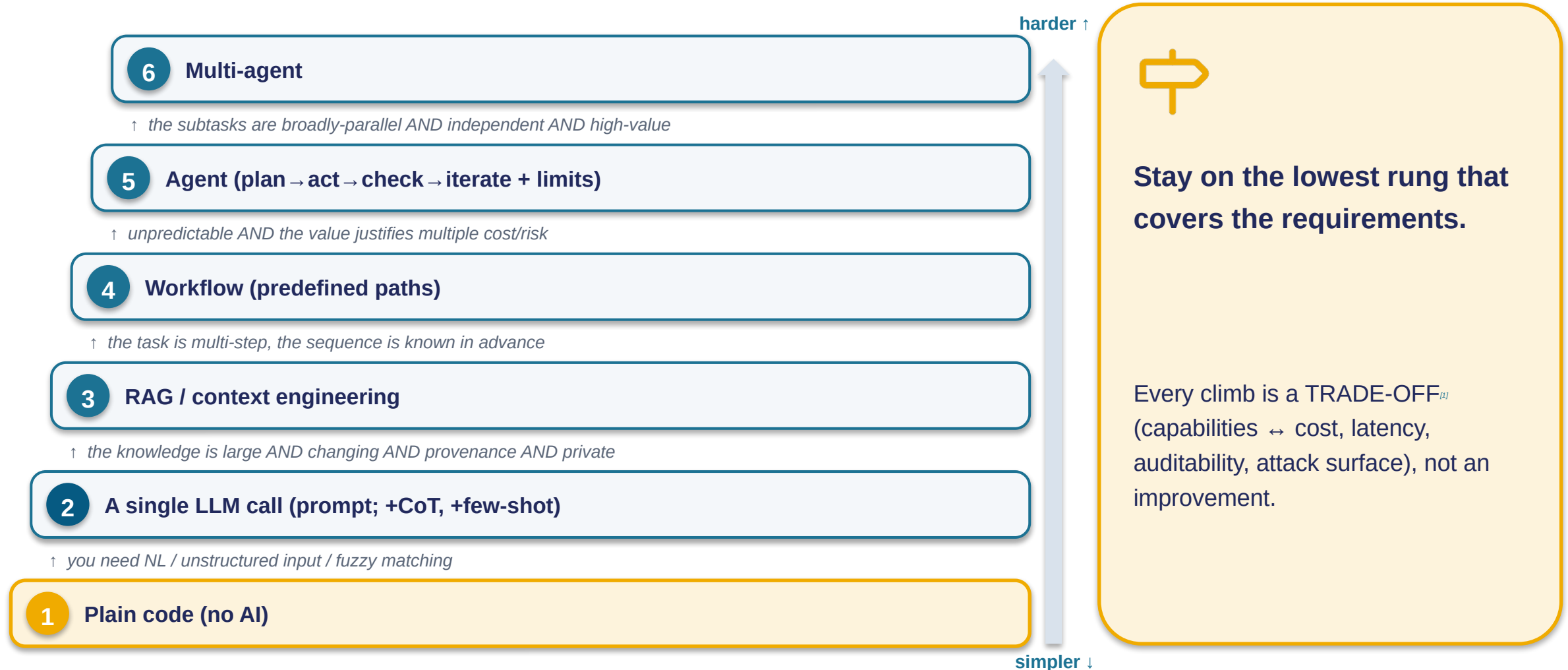
Section 3
Fine-tune

Section 4
Agents

Section 5
Framework

The ladder of architectural complexity.

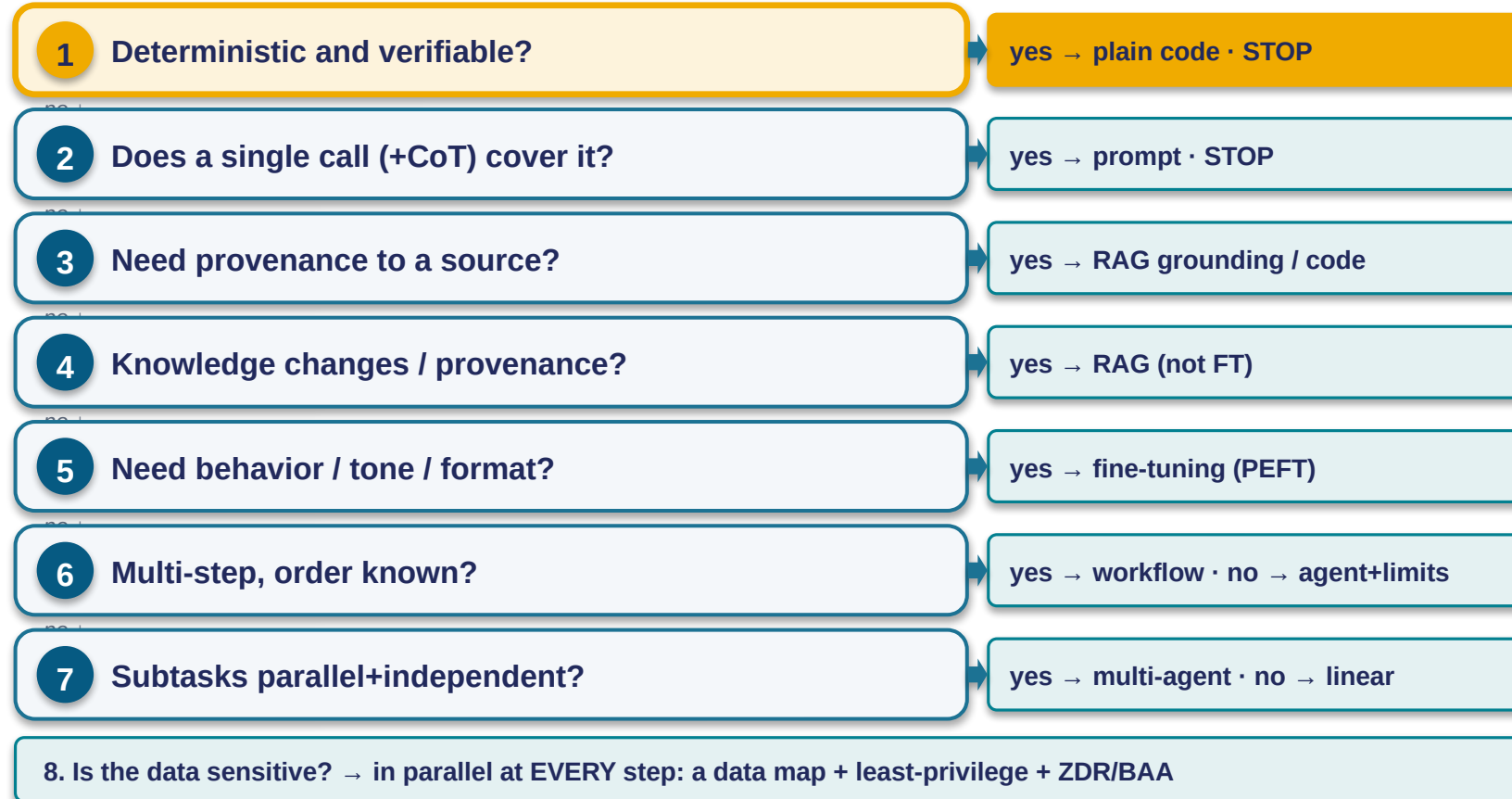
Stay on the lowest rung; climb only for a task requirement.



^[1] Anthropic — Building Effective Agents (the ladder rule) · The lowest rung is “plain code, no AI”: the ladder starts with the question “is AI needed at all?”

The decision plan: a route of questions, not a sum of points.

Walk the task from top to bottom; stop at the first question that triggers^[1].



Example (task A)

“2000 regulations, updated weekly, an answer with a link to the clause” → question 4 (changes+provenance) is decisive → RAG with strict grounding. Not FT (goes stale), not code (needs NL).

Warm-up (task B)

“a bot over ~150 FAQs, updated once a quarter” — walk the route yourself; discussed in the seminar.

Route priority: if the task is deterministic and verifiable — plain code, STOP here^[2]. AI would only add nondeterminism, cost, latency, and a surface for prompt injection.

The agent starter kit — and when to complicate it.

The same “don't complicate without a requirement” ladder, applied one level down — to the harness of a single agent, not the whole system.



Default — a thin agent

- one instruction file
- flat memory
- NO subagents
- a minimal set of skills
- minimal MCP access

The burden of proof is on complication, not on simplicity.



A memory backend — when:

history has outgrown the context OR you need structured retrieval over facts (not “the last N messages”). The same criterion as the prompt → RAG transition.



Subagents — when:

a subtask requires a separate window (don't clutter the context) OR isolation of untrusted work (least-privilege).



More MCP access — when:

a specific task requires a specific tool — not “just in case.” Each connection is a new trust boundary.

The same principle as the architecture ladder — applied to the harness of a single agent: the presence paradox showed that even an instruction file “as a ritual” doesn't work. Complicate only for a concrete, verifiable trigger.

The human validator + the MIT NANDA lesson.



The agent does — the human checks the result and the facts

*against an independent source of truth, NOT by the plausibility of the reasoning^[1]
(self-rationale ≠ control — the faithfulness lesson).*

Degree of autonomy

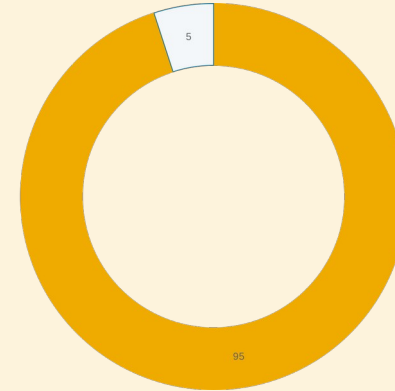
from “the human presses a button” to “the agent notifies after the fact”

Trust domain

reading (easy to roll back) vs writing · reversible vs irreversible

Continuous monitoring

quality metrics all the time, not a one-time check at the start



~95%

of corporate GenAI pilots with no measurable return on investment — the root is in the learning gap and integration failure, not in model quality^[2]

“Launching AI” ≠ “getting value.” What decides it is architectural-integration discipline. Sometimes the right answer is the simplest architecture or no AI.

Next — industries. We start with software development.

<> Lecture 4

AI in software development

the same apparatus — on a developer's concrete tasks: the same coding agents we covered, from the side of engineering practice.



Assignment — Seminar 3

“Architectural choice: chat / agent / RAG / API — 3 cases”

Apply the checklist to 3 cases; for each: architecture + ≥ 2 reasons by the axes + ≥ 1 condition that would change the choice + a criterion for “here it would NOT be so.”
Task B is a warm-up.

The apparatus is assembled — run the checklist on every industry lecture.



Photo: a developer at an IDE · Wikimedia · CC-BY-SA

Q&A

Thank you

[instructor contact details — fill in before the lecture]